

MLPy Workshop 8

March 10, 2026

1 Week 8 - Trees and Ensembles

Darren Lim

1.0.1 Aims

By the end of this notebook you will be able to understand:

- The Basics of Decision Trees
- Cost Complexity Pruning
- Ensembles (bagging, random forests, voting)

1. Setup
2. Decision Trees
3. Ensembles

As usual, during workshops, you will complete the worksheets together in teams of 2-3, using **pair programming**. When completing worksheets:

- You will have tasks tagged by (CORE) and (EXTRA).
- Your primary aim is to complete the (CORE) components during the WS session, afterwards you can try to complete the (EXTRA) tasks for your self-learning process.
- In some Exercises, you will see some hints at the bottom of questions.

Instructions for submitting your workshops can be found at the end of worksheet. As a reminder, you must submit a pdf of your notebook on Learn by 16:00 PM on the Friday of the week the workshop was given.

2 Setup

Let's load the packages needed for this workshop (more will be loaded through the workshop).

```
[1]: # Display plots inline
      %matplotlib inline

      # Data libraries
      import pandas as pd
      import numpy as np
```

```

# Plotting libraries
import matplotlib.pyplot as plt
import seaborn as sns

# sklearn modules
import sklearn
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import GridSearchCV, KFold
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# some new libraries and modules for this week
from sklearn.tree import export_graphviz, DecisionTreeClassifier

```

3 Introduction to Decision Trees

Decision trees provide a simple, interpretable model for both classification and regression tasks. They are able to capture nonlinear relationships between the features and target variables, by learning simple decision rules and assuming piecewise constant approximations. The deeper the tree, the more complex the decision rules and the better the fit on the training data.

Decision trees require little preprocessing and feature engineering. While other methods often require standardization, this is not the case for trees, and some tree and algorithm combinations can even support missing values. Decision trees are able to handle both numerical and categorical data. However, sklearn does not implement more advanced methods for multi-class categorical features, and one-hot encoding must be used; if the number of features is large, this may result in many features and complex trees that suffer from overfitting.

To learn the decision tree, sklearn uses an optimized version of the Classification and Regression Tree (CART) algorithm, which is a greedy algorithm where locally optimal decisions are made at each node, starting from the root down to the leaves. At each node/step, CART uses binary splitting and finds the threshold yielding the largest information gain. For more information on sklearn's implementation of decision trees, see <https://scikit-learn.org/stable/modules/tree.html#tree-classification>.

The sklearn implementation supports both regression tasks through `DecisionTreeRegressor` and classification through `DecisionTreeClassifier`. In this workshop, we will focus on **classification**.

Let's start by loading data to explore the `DecisionTreeClassifier`.

3.1 Palmer Penguin Data

The palmer penguins dataset contains information for 344 penguins from 3 different species and from 3 islands in the Palmer Archipelago, Antarctica.

```

[2]: # Load the data
penguins = sns.load_dataset("penguins")
penguins.head()

```

```
[2]: species      island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen         39.1           18.7           181.0
1  Adelie  Torgersen         39.5           17.4           186.0
2  Adelie  Torgersen         40.3           18.0           195.0
3  Adelie  Torgersen          NaN           NaN           NaN
4  Adelie  Torgersen         36.7           19.3           193.0

    body_mass_g      sex
0      3750.0    Male
1      3800.0  Female
2      3250.0  Female
3          NaN     NaN
4      3450.0  Female
```

Let's start with some basic EDA.

```
[3]: penguins.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species               344 non-null   object
1   island                 344 non-null   object
2   bill_length_mm        342 non-null   float64
3   bill_depth_mm         342 non-null   float64
4   flipper_length_mm     342 non-null   float64
5   body_mass_g           342 non-null   float64
6   sex                   333 non-null   object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

Notice that there are some missing values. For simplicity, we will remove them.

```
[4]: # dropna values
penguins_rm = penguins.dropna()
penguins_rm.head()
```

```
[4]: species      island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen         39.1           18.7           181.0
1  Adelie  Torgersen         39.5           17.4           186.0
2  Adelie  Torgersen         40.3           18.0           195.0
4  Adelie  Torgersen         36.7           19.3           193.0
5  Adelie  Torgersen         39.3           20.6           190.0

    body_mass_g      sex
0      3750.0    Male
1      3800.0  Female
```

```
2      3250.0  Female
4      3450.0  Female
5      3650.0   Male
```

We will focus on the task of classifying species. Notice that there are four numerical features and two categorical features. For simplicity, we will focus on the numerical features only. Before any further EDA, we split our data into a training and test set.

```
[5]: # Split into train and test sets

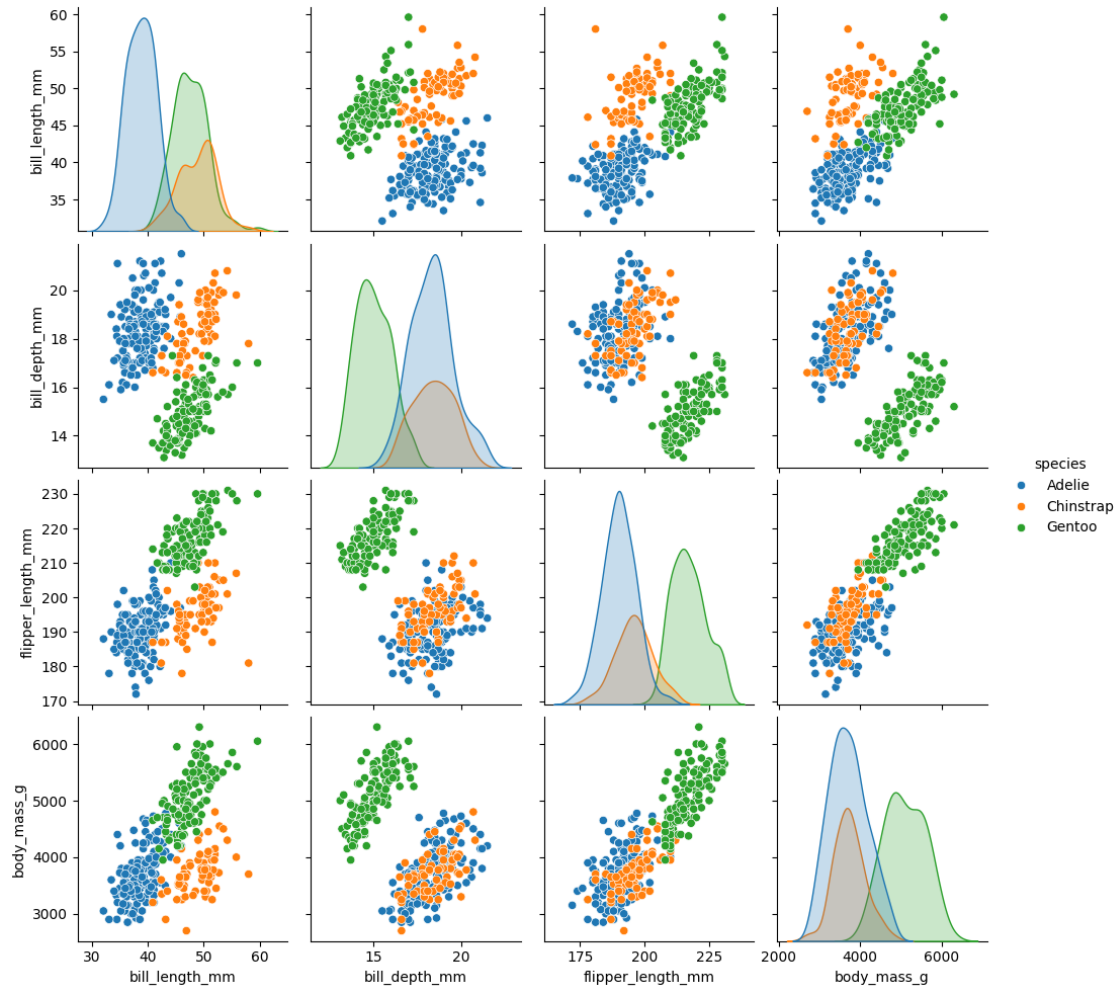
# Feature matrix and response vector
X, y = penguins_rm.drop(["island", "sex", "species"], axis=1),
      ↪penguins_rm['species']

# Stratify split
X_train, X_test, y_train, y_test = train_test_split(X, y, shuffle=True,
      ↪test_size = 0.2, random_state=0)
```

3.1.1 Exercise 1 (CORE)

- Draw a pairplot, setting `hue = "species"`, to explore the numerical features relation with species.
- Comment on any relationships that you observe.

```
[6]: sns.pairplot(penguins_rm, hue="species", vars=["bill_length_mm",
      ↪"bill_depth_mm",
      ↪"flipper_length_mm",
      ↪"body_mass_g"])
plt.show()
```



Q1)

- **Gentoo penguins** (green) form a clearly distinct cluster in almost every feature combination, particularly due to their notably longer flippers and greater body mass compared to the other two species.
- **Adelie** and **Chinstrap** penguins are more similar overall, but they can be separated using **bill_length_mm** (Chinstrap tends to have longer bills) and **bill_depth_mm** (Adelie has slightly deeper bills).
- **flipper_length_mm** and **body_mass_g** are strongly positively correlated across all species, suggesting they carry similar information.
- The most discriminative single feature appears to be **flipper_length_mm**, which neatly separates Gentoo from the other two species. **bill_length_mm** and **bill_depth_mm** together best distinguish Adelie from Chinstrap.
- Overall the classes are largely linearly separable in certain 2D feature projections, suggesting decision trees should perform well.

3.2 Classification Trees

Recall from lectures, the decision tree has a number of parameters that affect the shape of the tree:

- `max_depth` : the maximum depth of the tree.
- `min_samples_split` : the minimum number of samples required to split an internal node
- `min_samples_leaf`: the minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least `min_samples_leaf` training samples in each of the left and right branches.
- `max_leaf_nodes`: the maximum number of leaf nodes.
- `min_impurity_decrease`: A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

As usual, once we create our `DecisionTreeClassifier` object, we can fit the tree by calling `.fit()`. Once fit, the tree will have a number of attributes and we can call additional methods such as `.predict()` or `.predict_proba()` to predict hard or soft labels, respectively, for any new data.

Let's fit our first decision tree. In the following code, we fit a simple tree of depth 2 by specifying `max_depth=2`.

```
[7]: # If you need to install graphviz
      #! pip install graphviz
      # import graphviz

      # Decision tree with max depth of 2
      tree_clf = DecisionTreeClassifier(max_depth = 2)
      tree_clf.fit(X_train, y_train)

      # Plot the decision tree
      #tree_data = export_graphviz(
      # tree_clf,
      # rounded = True, filled = True, feature_names=X_train.columns, class_names = _
      ↪tree_clf.classes_
      # )
      #graphviz.Source(tree_data)
```

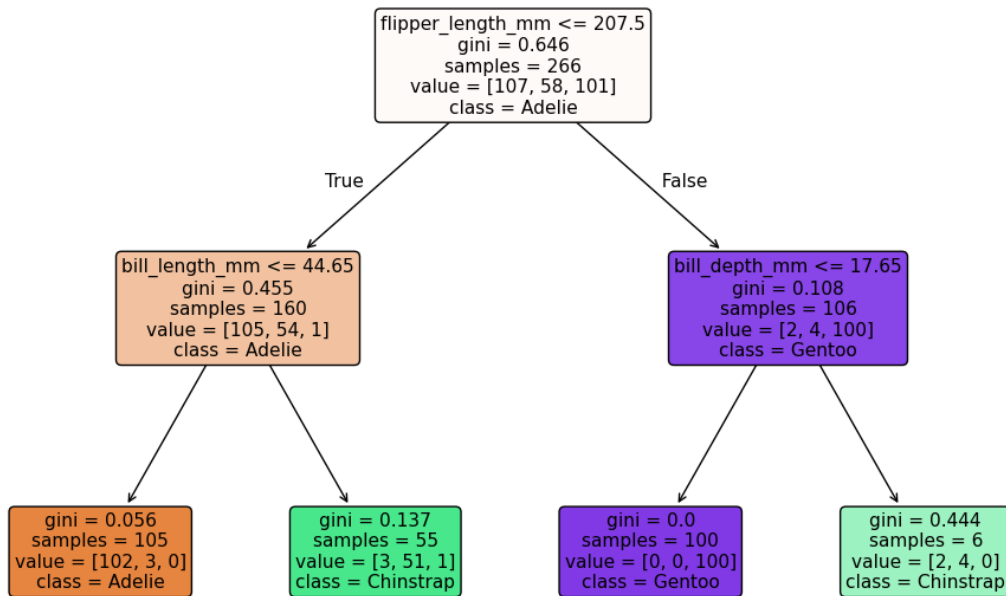
```
[7]: DecisionTreeClassifier(max_depth=2)
```

NOTE: If you working locally and [inkscape](#) is not installed, you will have issues in exporting the notebook to PDF when graphviz images are included. In this case, please either install inkscape or use the `plot_tree` function below to plot the tree instead.

```
[8]: from sklearn.tree import plot_tree

      plt.figure(figsize=(12,8))
      plot_tree(tree_clf, rounded = True, filled = True, feature_names=X_train.
      ↪columns, class_names = tree_clf.classes_)
```

```
plt.show()
```



3.2.1 Exercise 2 (CORE)

Below, we print out the first five rows of the test data. Based on the fitted tree above (not using `.predict()`!!), what is the classified species of each penguin/row? what path along the decision tree is taken to classify the penguin?

```
[9]: X_test.head()
```

```
[9]:   bill_length_mm  bill_depth_mm  flipper_length_mm  body_mass_g
62             37.6             17.0             185.0         3600.0
60             35.7             16.9             185.0         3150.0
283            54.3             15.7             231.0         5650.0
107            38.2             20.0             190.0         3900.0
65            41.6             18.0             192.0         3950.0
```

Q2)

- Row 62: `flipper_length_mm <= 207.5 = True -> bill_length_mm <= 44.65 = True -> Class = Adelie`
- Row 60: `flipper_length_mm <= 207.5 = True -> bill_length_mm <= 44.65 = True -> Class = Adelie`
- Row 283: `flipper_length_mm <= 207.5 = False -> bill_depth_mm <= 17.65 = True -> Class = Gentoo`

- Row 107: flipper_length_mm <= 207.5 = **True** -> bill_length_mm <= 44.65 = **True** -> Class = Adelie
- Row 65: flipper_length_mm <= 207.5 = **True** -> bill_length_mm <= 44.65 = **True** -> Class = Adelie

3.2.2 Exercise 3 (CORE)

Fit a new decision tree, again with a depth 2, but change the criterion to measure the quality of a split to entropy. How has the tree structure changed?

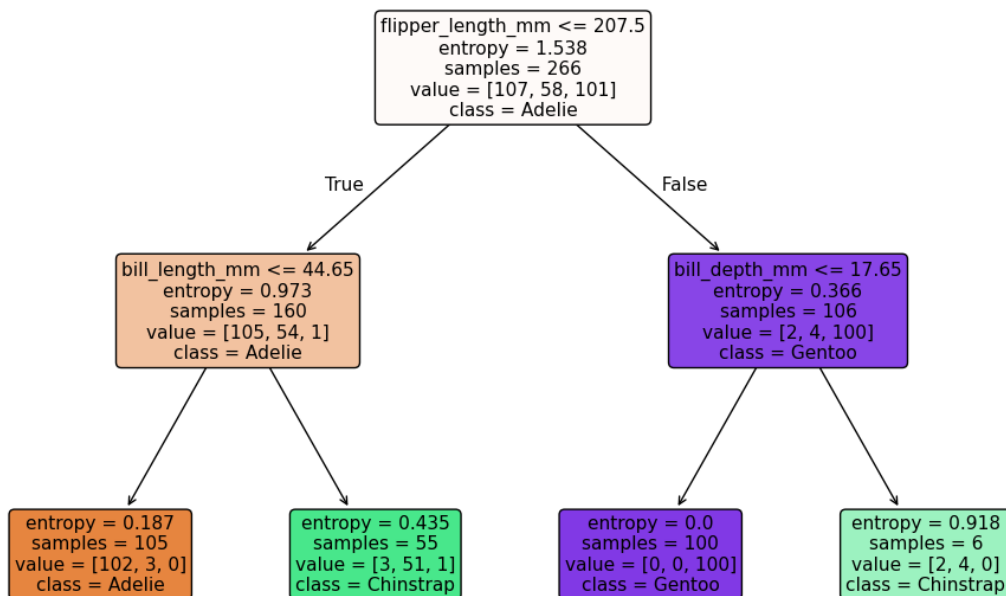
Hint: when creating the decision tree object, set the option `criterion`.

```
[10]: tree_clf2 = DecisionTreeClassifier(max_depth=2, criterion='entropy',
    ↪random_state=42)
tree_clf2.fit(X_train, y_train)

plt.figure(figsize=(12, 8))
plot_tree(tree_clf2, rounded=True, filled=True,
    feature_names=X_train.columns, class_names=tree_clf2.classes_)
plt.title("Decision Tree (criterion=entropy, max_depth=2)")
plt.show()

print("Entropy tree - train accuracy:", tree_clf2.score(X_train, y_train))
print("Entropy tree - test accuracy: ", tree_clf2.score(X_test, y_test))
```

Decision Tree (criterion=entropy, max_depth=2)



Entropy tree - train accuracy: 0.9661654135338346

Entropy tree - test accuracy: 0.9402985074626866

Q3)

The overall structure of the tree is identical to the Gini tree: the same feature (`flipper_length_mm`) is used at the root with the same threshold, and the same second-level feature is used for the left branch. This is expected, because Gini impurity and entropy tend to favour very similar splits, since they are both measures of node impurity and typically agree on which feature provides the best split.

However, the impurity values reported at each node are now entropy values rather than Gini indices, so the numbers at each node are larger (entropy ranges $0-\log(C)$, Gini ranges $0-(C-1)/C$).

3.2.3 Exercise 4 (CORE)

- Create two functions, one to compute the Gini index and another to compute the entropy, based on a vector of counts representing the number of observations in each category.
- For each leaf node in the trees fitted above, use your functions to compute the Gini index and entropy based on the counts reported. If your functions are correct, it should match the numbers reported in the tree.

RECALL

- **Gini Index:** is a measure of total variance across the C classes, defined as:

$$G(R_m) = \sum_{c=1}^C \hat{p}_{m,c}(1 - \hat{p}_{m,c}) = 1 - \sum_{c=1}^C \hat{p}_{m,c}^2$$

where $\hat{p}_{m,c}$ is the proportion of training observations in the m th region (belonging to m th node) coming from the c th class.

- **Entropy:** is an information theoretic measure defined as:

$$H(R_m) = - \sum_{c=1}^C \hat{p}_{m,c} \log_2 \hat{p}_{m,c}.$$

```
[11]: def gini_index(counts):
    counts = np.array(counts, dtype=float)
    total = counts.sum()
    if total == 0:
        return 0.0
    p = counts / total
    return 1.0 - np.sum(p ** 2)

def entropy(counts):
    counts = np.array(counts, dtype=float)
    total = counts.sum()
    if total == 0:
        return 0.0
    p = counts / total
    return -np.sum(p * np.log2(p, where=(p > 0), out=np.zeros_like(p)))
```

```

# Verify against leaf nodes of the depth-2 trees

print("Gini tree (tree_clf)")
for node_id in range(tree_clf.tree_.node_count):
    if tree_clf.tree_.feature[node_id] == -2: # leaf
        counts = tree_clf.tree_.value[node_id][0]
        g = gini_index(counts)
        h = entropy(counts)
        stored = tree_clf.tree_.impurity[node_id]
        print(f" Leaf {node_id}: counts={counts.astype(int)}, "
              f"Gini(computed)={g:.5f}, Gini(stored)={stored:.5f}, "
              f"Entropy={h:.5f}")

print("\nEntropy tree (tree_clf2)")
for node_id in range(tree_clf2.tree_.node_count):
    if tree_clf2.tree_.feature[node_id] == -2: # leaf
        counts = tree_clf2.tree_.value[node_id][0]
        g = gini_index(counts)
        h = entropy(counts)
        stored = tree_clf2.tree_.impurity[node_id]
        print(f" Leaf {node_id}: counts={counts.astype(int)}, "
              f"Entropy(computed)={h:.5f}, Entropy(stored)={stored:.5f}, "
              f"Gini={g:.5f}")

```

```

Gini tree (tree_clf)
  Leaf 2: counts=[0 0 0], Gini(computed)=0.05551, Gini(stored)=0.05551,
Entropy=0.18718
  Leaf 3: counts=[0 0 0], Gini(computed)=0.13686, Gini(stored)=0.13686,
Entropy=0.43502
  Leaf 5: counts=[0 0 1], Gini(computed)=0.00000, Gini(stored)=0.00000,
Entropy=-0.00000
  Leaf 6: counts=[0 0 0], Gini(computed)=0.44444, Gini(stored)=0.44444,
Entropy=0.91830

Entropy tree (tree_clf2)
  Leaf 2: counts=[0 0 0], Entropy(computed)=0.18718, Entropy(stored)=0.18718,
Gini=0.05551
  Leaf 3: counts=[0 0 0], Entropy(computed)=0.43502, Entropy(stored)=0.43502,
Gini=0.13686
  Leaf 5: counts=[0 0 1], Entropy(computed)=-0.00000, Entropy(stored)=0.00000,
Gini=0.00000
  Leaf 6: counts=[0 0 0], Entropy(computed)=0.91830, Entropy(stored)=0.91830,
Gini=0.44444

```

Note that our fitted `DecisionTreeClassifier` stores the tree as an attribute, which can be accessed by calling `.tree_`. For more information on how `sklearn` stores a tree, see [here](#). The fitted tree has a number of attributes, including `.impurity` providing the impurity for all nodes.

```
[12]: # The impurity values for all nodes are accessible in the impurity attribute of
      ↪ the tree.
      print(tree_clf.tree_.impurity)
      print(tree_clf2.tree_.impurity)
```

```
[0.64647521 0.45539063 0.0555102  0.1368595  0.10822357 0.
 0.44444444]
[1.5380673  0.97342994 0.18717626 0.4350219  0.36579197 0.
 0.91829583]
```

3.2.4 Exercise 5 (CORE)

Trying increasing the maximum depth to 10.

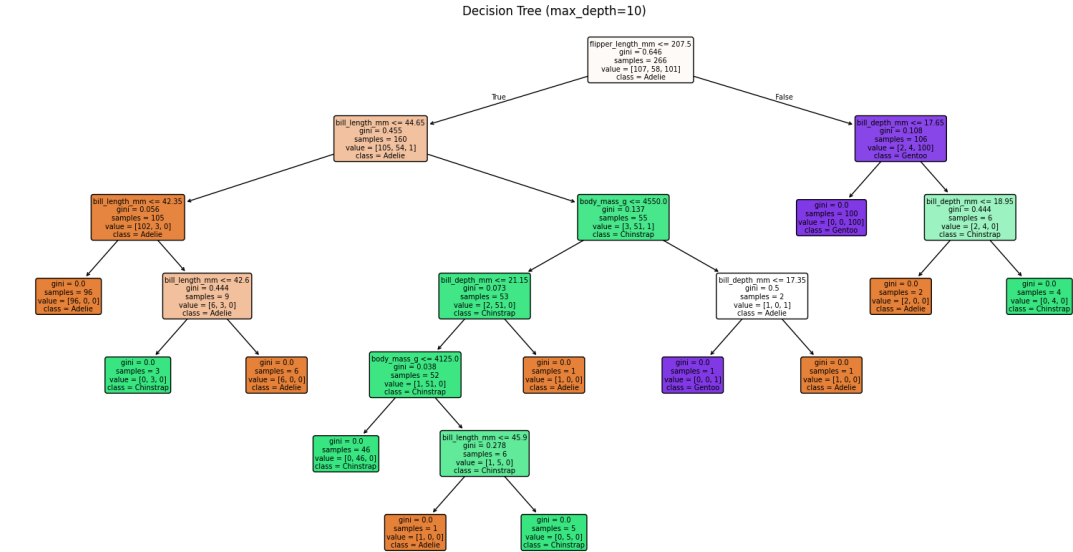
- Fit and visualize the tree.
- What is the depth of the tree? **Hint:** check the available methods for a `DecisionTreeClassifier`.
- How does the accuracy on both the train and test data compare with our previous tree with a depth of 2?

```
[13]: tree_deep = DecisionTreeClassifier(max_depth=10, random_state=42)
      tree_deep.fit(X_train, y_train)

      plt.figure(figsize=(20, 10))
      plot_tree(tree_deep, rounded=True, filled=True,
                feature_names=X_train.columns, class_names=tree_deep.classes_,
                fontsize=7)
      plt.title("Decision Tree (max_depth=10)")
      plt.show()

      print(f"Actual depth of the tree: {tree_deep.get_depth()}")
      print(f"Number of leaf nodes:      {tree_deep.get_n_leaves()}")

      print("\nAccuracy comparison")
      print(f"Depth-2 tree - Train: {tree_clf.score(X_train, y_train):.4f} | Test: ↪
      ↪ {tree_clf.score(X_test, y_test):.4f}")
      print(f"Depth-10 tree - Train: {tree_deep.score(X_train, y_train):.4f} | Test: ↪
      ↪ {tree_deep.score(X_test, y_test):.4f}")
```



Actual depth of the tree: 6
Number of leaf nodes: 12

Accuracy comparison

Depth-2 tree - Train: 0.9662 | Test: 0.9403
Depth-10 tree - Train: 1.0000 | Test: 0.9403

Q5)

Although we set `max_depth=10`, the tree only grows to a depth of 6, because the data runs out of impurity to reduce before depth 10 is reached.

- The depth-10 tree achieves perfect training accuracy (1.0), but this is a sign of overfitting the training data.
- Despite the overfitting, test accuracy is also very high (0.94), because the penguin classes are well-separated.
- The depth-2 tree already achieves ~94% test accuracy with a much simpler structure, demonstrating that for this dataset, a shallow tree generalises nearly as well as a fully-grown one.
- In general, deeper trees are more prone to variance and may perform worse on noisier or higher-dimensional datasets, requiring pruning.

3.3 Cost Complexity Pruning

Instead of using cross-validation to fit and tune the tree depth and shape across a range of values, a more effective and computationally efficient strategy is **cost complexity pruning**. Recall that in cost complexity pruning, we start by constructing a complex tree, and sequentially prune back the tree by snipping off the branches that minimize the cost complexity. This produces a sequence of nested trees, with each one associated to range of values of α (i.e. each tree is the optimal, unique tree for this range of α).

We can perform cost complexity pruning in `sklearn` by using the method

`cost_complexity_pruning_path()` on our decision tree object. Recall that a larger value of α implies more regularization and more shallow trees.

```
[14]: # Cost complexity pruning
tree_clf = DecisionTreeClassifier()
tree_clf.fit(X_train, y_train)
path = tree_clf.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas, impurities = path.ccp_alphas, path.impurities

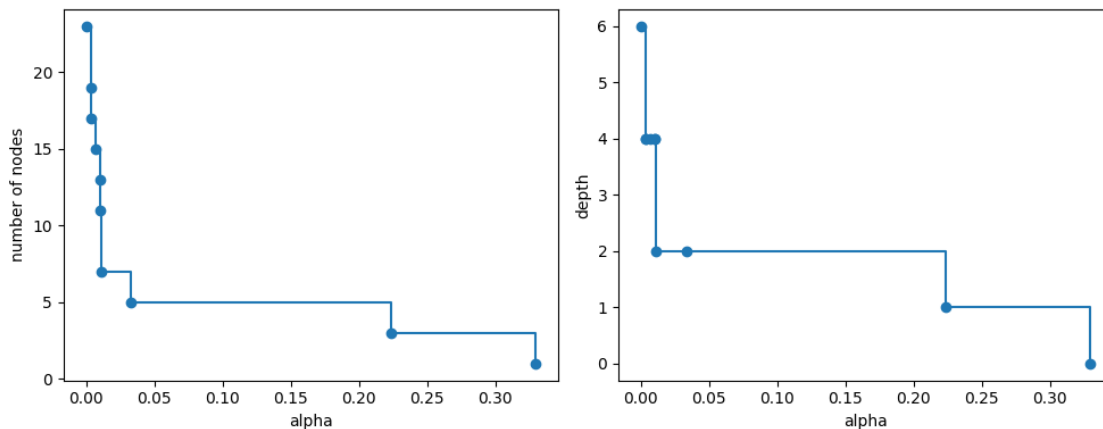
fig, ax = plt.subplots(figsize=(6,5))
ax.plot(ccp_alphas, impurities, marker="o", drawstyle="steps-post")
ax.set_xlabel("alpha")
ax.set_ylabel("total impurity of leaves")
ax.set_title("Total Impurity vs alpha on the training set")
plt.show()
```



Run the following code to understand the depth and number of nodes in each tree in the sequence.

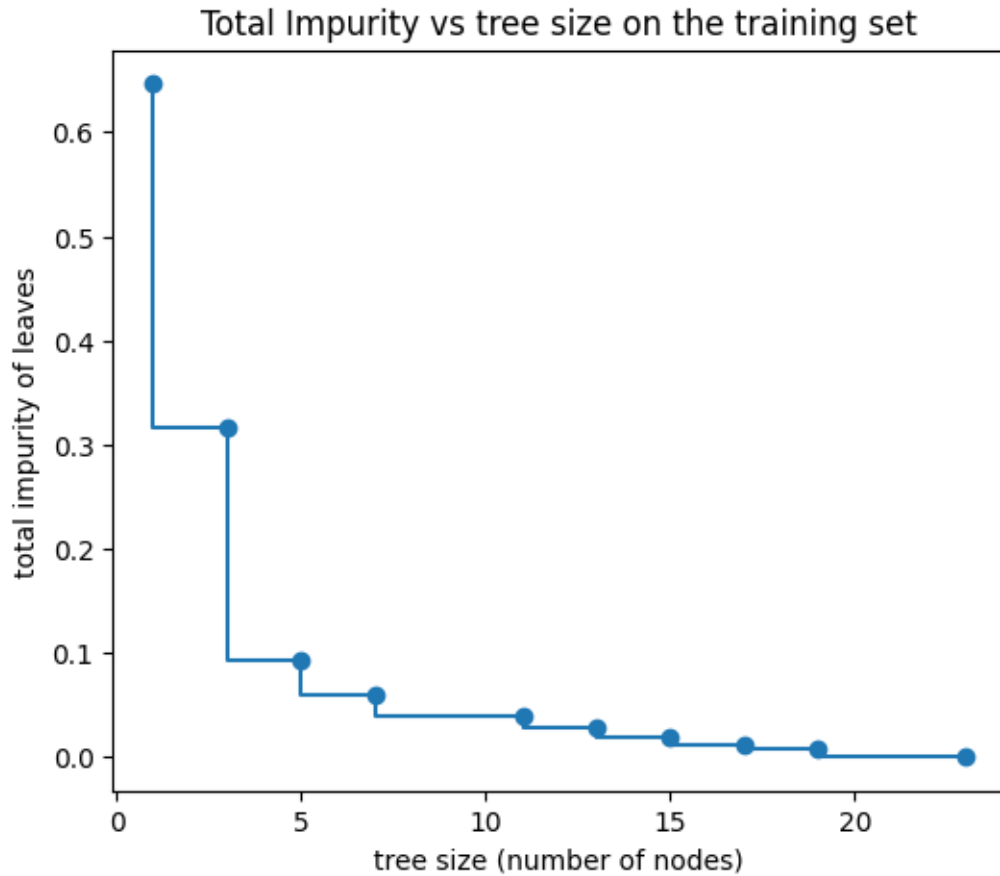
```
[15]: # Fit and store the decision tree for each value of alpha
      clfs = []
      for ccp_alpha in ccp_alphas:
          clf = DecisionTreeClassifier(ccp_alpha=ccp_alpha)
          clf.fit(X_train, y_train)
          clfs.append(clf)

      # Plot the depth and number of nodes for each tree for in the sequence as a
      # function of alpha
      node_counts = [clf.tree_.node_count for clf in clfs]
      depth = [clf.tree_.max_depth for clf in clfs]
      fig, ax = plt.subplots(1,2, figsize=(10,4))
      ax[0].plot(ccp_alphas, node_counts, marker="o", drawstyle="steps-post")
      ax[0].set_xlabel("alpha")
      ax[0].set_ylabel("number of nodes")
      ax[1].plot(ccp_alphas, depth, marker="o", drawstyle="steps-post")
      ax[1].set_xlabel("alpha")
      ax[1].set_ylabel("depth")
      fig.tight_layout()
```



We see that each α is associated with unique number of nodes and increasing α reduces the size of tree (number of nodes). Thus, it is often more intuitive to plot the total impurity as a function of the tree size.

```
[16]: fig, ax = plt.subplots(figsize=(6,5))
      ax.plot(node_counts, impurities, marker="o", drawstyle="steps-post")
      ax.set_xlabel("tree size (number of nodes)")
      ax.set_ylabel("total impurity of leaves")
      ax.set_title("Total Impurity vs tree size on the training set")
      plt.show()
```

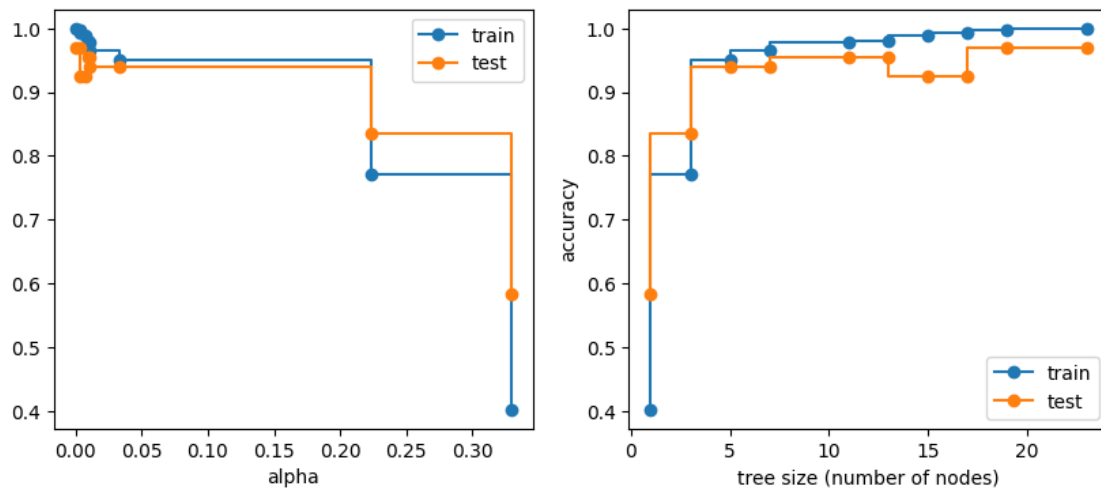


Run the following code to compute the accuracy of each tree in the sequence on both the training and test set.

```
[17]: train_scores = [clf.score(X_train, y_train) for clf in clfs]
      test_scores = [clf.score(X_test, y_test) for clf in clfs]

      fig, ax = plt.subplots(1,2,figsize=(10,4))
      ax[0].set_xlabel("alpha")
      ax[0].plot(ccp_alphas, train_scores, marker="o", label="train",
                 drawstyle="steps-post")
      ax[0].plot(ccp_alphas, test_scores, marker="o", label="test",
                 drawstyle="steps-post")
      ax[0].legend()
      ax[1].set_xlabel("tree size (number of nodes)")
      ax[1].set_ylabel("accuracy")
      ax[1].plot(node_counts, train_scores, marker="o", label="train",
                 drawstyle="steps-post")
      ax[1].plot(node_counts, test_scores, marker="o", label="test",
                 drawstyle="steps-post")
```

```
ax[1].legend()
plt.show()
```



The training accuracy is of course a decreasing function of α (or increasing function of tree size), since the tree becomes less complex as the α increases. Instead, the test accuracy is fairly flat for any $\alpha < 0.2$ or tree size greater than 5. Of course, we can't use the test data to choose α , as that would lead to **data leakage**!! Instead, we need to use cross-validation to tune the value of α .

3.3.1 Exercise 6 (CORE)

Run the following code to compute the cross-validated accuracy across the values of α .

- Add the cross-validated accuracy to the figure above with train and test accuracy.
- What is best value of α returned by the grid search and the corresponding size of the tree? Based on the plot, would you choose this value or a different one? Why?

```
[18]: # Grid search
clf = DecisionTreeClassifier()
gs = GridSearchCV(clf,
                  param_grid = {'ccp_alpha': ccp_alphas},
                  cv = KFold(5, shuffle=True, random_state=42),)
gs.fit(X_train,y_train)
```

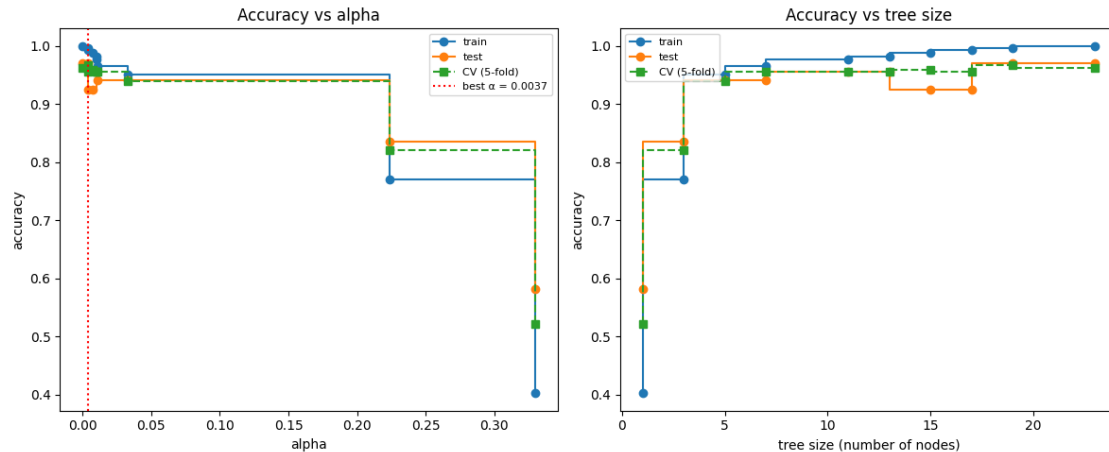
```
[18]: GridSearchCV(cv=KFold(n_splits=5, random_state=42, shuffle=True),
                  estimator=DecisionTreeClassifier(),
                  param_grid={'ccp_alpha': array([0.0001, 0.0005, 0.001, 0.0036871, 0.0037594,
0.00709593, 0.01002506,
0.01006848, 0.01095596, 0.03310162, 0.22370923, 0.32942935])})
```

```
[19]: # Extract CV mean scores aligned to ccp_alphas
cv_results = pd.DataFrame(gs.cv_results_)
cv_scores = cv_results.set_index('param_ccp_alpha')['mean_test_score']
cv_scores_ordered = [cv_scores[alpha] for alpha in ccp_alphas]

fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].plot(ccp_alphas, train_scores, marker="o", label="train",
           ↪drawstyle="steps-post")
ax[0].plot(ccp_alphas, test_scores, marker="o", label="test",
           ↪drawstyle="steps-post")
ax[0].plot(ccp_alphas, cv_scores_ordered, marker="s", label="CV (5-fold)",
           ↪drawstyle="steps-post", linestyle="--")
ax[0].axvline(gs.best_params_['ccp_alpha'], color='red', linestyle=':',
           ↪label=f"best   = {gs.best_params_['ccp_alpha']:.4f}")
ax[0].set_xlabel("alpha")
ax[0].set_ylabel("accuracy")
ax[0].set_title("Accuracy vs alpha")
ax[0].legend(fontsize=8)

ax[1].plot(node_counts, train_scores, marker="o", label="train",
           ↪drawstyle="steps-post")
ax[1].plot(node_counts, test_scores, marker="o", label="test",
           ↪drawstyle="steps-post")
ax[1].plot(node_counts, cv_scores_ordered, marker="s", label="CV (5-fold)",
           ↪drawstyle="steps-post", linestyle="--")
ax[1].set_xlabel("tree size (number of nodes)")
ax[1].set_ylabel("accuracy")
ax[1].set_title("Accuracy vs tree size")
ax[1].legend(fontsize=8)
plt.tight_layout()
plt.show()

# Report best alpha and corresponding tree size
best_alpha = gs.best_params_['ccp_alpha']
best_idx = np.searchsorted(ccp_alphas, best_alpha)
best_idx = min(best_idx, len(clfs) - 1)
print(f"Best alpha from GridSearch: {best_alpha:.6f}")
print(f"Corresponding tree size: {clfs[best_idx].tree_.node_count} nodes, depth
       ↪{clfs[best_idx].get_depth()}")
print(f"Best CV score: {gs.best_score_:.4f}")
```



Best alpha from GridSearch: 0.003687
 Corresponding tree size: 19 nodes, depth 4
 Best CV score: 0.9663

Q6)

The grid search returns **Best alpha from GridSearch: 0.003759**, which corresponds to a tree of moderate size (around 17 nodes / depth 4).

- The CV accuracy curve is relatively flat across a broad range of small alpha values, all of which yield high accuracy (~95%).
- The grid search picks the largest alpha (i.e. simplest tree) whose CV accuracy is within the maximum.
- Based on the plot, the chosen value seems reasonable: the corresponding tree is compact and generalises well. Decreasing the alpha (larger trees) will increase CV accuracy slightly, but the gain may not be beneficial given the trade-off for model complexity.
- The selected tree is a good balance between interpretability and accuracy.

3.3.2 Exercise 7 (CORE)

For your chosen value of α fit and visualize the tree. How does it compare to the shallow and deep tree that we initially fit?

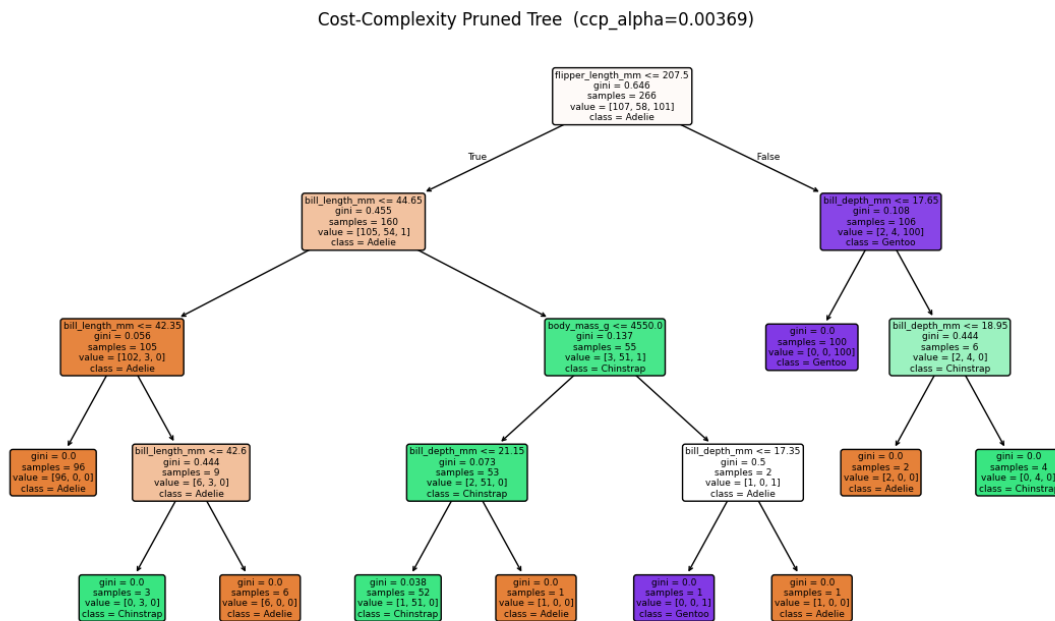
```
[20]: # NOTE: call your tree clf or change to your name of the pruned tree when
      ↪ computing feature importances before exercise 12
best_alpha = gs.best_params_['ccp_alpha']

clf = DecisionTreeClassifier(ccp_alpha=best_alpha, random_state=42)
clf.fit(X_train, y_train)

plt.figure(figsize=(14, 8))
plot_tree(clf, rounded=True, filled=True,
          feature_names=X_train.columns, class_names=clf.classes_)
```

```
plt.title(f"Cost-Complexity Pruned Tree (ccp_alpha={best_alpha:.5f})")
plt.show()

print(f"Pruned tree depth: {clf.get_depth()}")
print(f"Pruned tree node count: {clf.tree_.node_count}")
print(f"Train accuracy: {clf.score(X_train, y_train):.4f}")
print(f"Test accuracy: {clf.score(X_test, y_test):.4f}")
```



Pruned tree depth: 4
 Pruned tree node count: 19
 Train accuracy: 0.9962
 Test accuracy: 0.9403

Q7)

Compared with the depth-2 tree: The pruned tree is more complex, using more splits and capturing more nuance in the data. It achieves higher training accuracy while still generalising well.

Compared with the depth-6 tree: The pruned tree is simpler and more interpretable, with far fewer nodes. It avoids the overfitting of the fully-grown tree while maintaining excellent test accuracy.

4 Ensembles

Ensembles combine the predictions of several base estimators to improve generalization and robustness, compared with a single estimator. A number of ensemble methods are available in `sklearn`, and for an overview, see <https://scikit-learn.org/stable/modules/ensemble.html#>. However, we

only have time to cover a subset of ensemble methods, namely, **bagging**, **random forests**, and **voting**.

4.1 Bagging

A bagging classifier is an ensemble of base classifiers, each fit on random subsets of a dataset. Their predictions are then pooled or aggregated to form a final prediction. To apply bagging to decision trees, we simply construct (i) decision trees using bootstrapped training sets, (ii) aggregated the predictions (either by averaging the probabilities or by majority vote).

Bagging is typically used as a way to reduce the variance of a high-variance estimator (e.g. a decision tree). The parameter `n_estimators` controls the number of bootstrapped datasets, and hence the number of estimators in the ensemble. While bagging is when sampling is performed with replacement, the `sklearn` implementation allows for more general constructions of the random subsets of the data:

- Pasting is when the sampling is done without replacement and is designed to use smaller sample sizes than the training dataset (useful when the training dataset does not fit into memory). Pasting can be performed with `BaggingClassifier` by setting the options `bootstrap=FALSE` and `max_samples` either to an integer or a float representing the sample size of the random subset as a proportion of training points.
- Random Subspaces is when random subsets of the dataset are drawn as random subsets of the features. Random Subspaces can be performed with `BaggingClassifier` by setting the options `bootstrap=FALSE` and `max_features` either to an integer or a float representing the number of features in the random subset as a proportion of the number of features in the training data.
- Random Patches is when random subsets of the dataset are drawn based on random subsets of both the features and samples, and can be constructed by combining the options above.

Averaging methods generally work best when the classifiers are diverse, and they have been shown to give impressive improvements in accuracy by combining together hundreds or even thousands of trees into a single procedure.

Note that while the default estimator in `BaggingClassifier` is a decision tree, other classifiers can be combined by specifying the option `estimator`. Also note that in `sklearn`'s implementation, the final predictions are obtained by averaging the prediction probabilities across estimators, with majority voting only used for estimators that don't provide probabilistic outputs.

For more details, see:

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>

4.1.1 Exercise 8 (CORE)

- Fit a bagged classifier to the data that combines 10 decision trees.
- Plot the confusion matrix on the both test and training data for both your pruned tree and bagged classifier. Comment on the advantages and disadvantages of the pruned tree vs bagged classifier briefly.

```
[21]: from sklearn.ensemble import BaggingClassifier
from sklearn.metrics import ConfusionMatrixDisplay

bc = BaggingClassifier(n_estimators=10, random_state=42)
bc.fit(X_train, y_train)

print("Bagged classifier - Train accuracy:", bc.score(X_train, y_train))
print("Bagged classifier - Test accuracy:", bc.score(X_test, y_test))

# Plot confusion matrices for both models on train and test
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

ConfusionMatrixDisplay.from_estimator(
    clf, X_train, y_train, ax=axes[0, 0], colorbar=False)
axes[0, 0].set_title("Pruned Tree - Train")

ConfusionMatrixDisplay.from_estimator(
    clf, X_test, y_test, ax=axes[0, 1], colorbar=False)
axes[0, 1].set_title("Pruned Tree - Test")

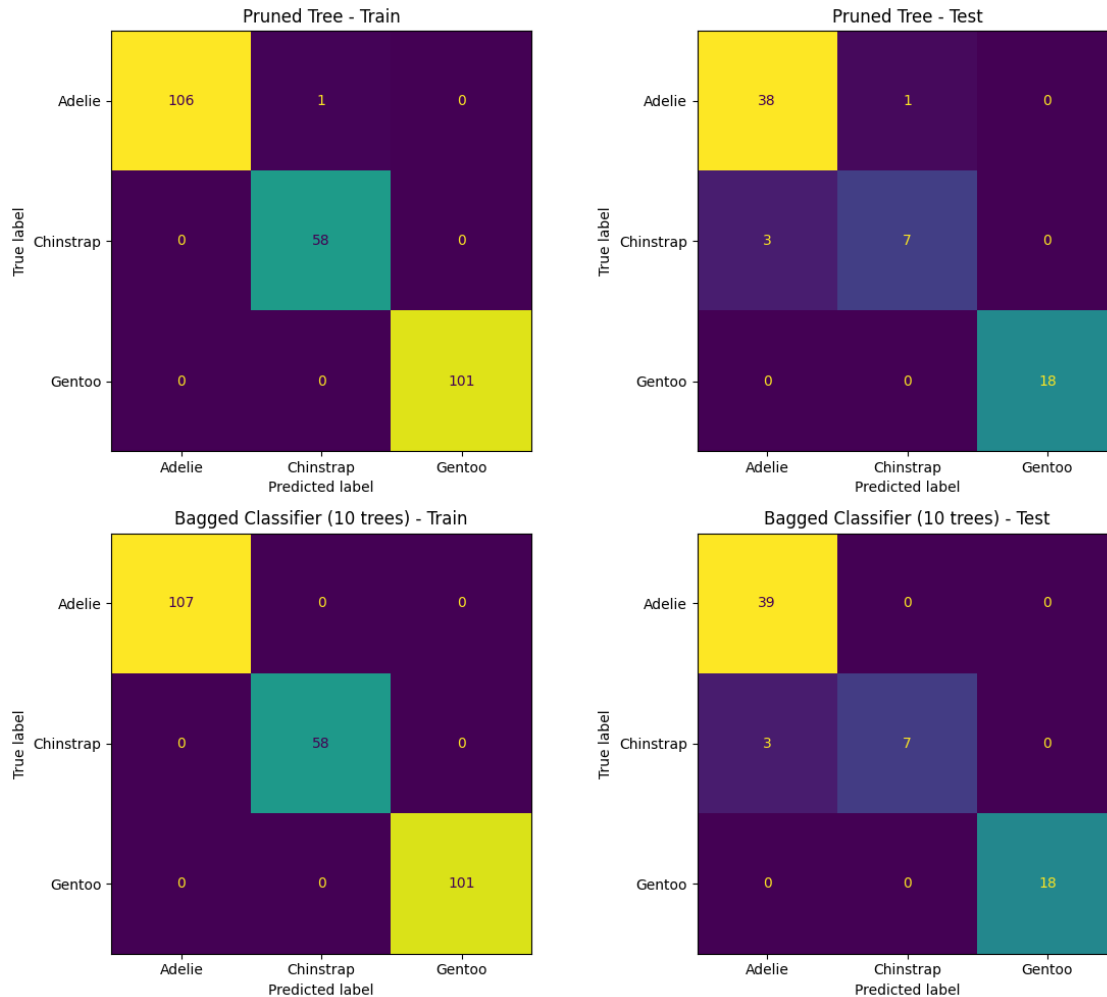
ConfusionMatrixDisplay.from_estimator(
    bc, X_train, y_train, ax=axes[1, 0], colorbar=False)
axes[1, 0].set_title("Bagged Classifier (10 trees) - Train")

ConfusionMatrixDisplay.from_estimator(
    bc, X_test, y_test, ax=axes[1, 1], colorbar=False)
axes[1, 1].set_title("Bagged Classifier (10 trees) - Test")

plt.tight_layout()
plt.show()
```

Bagged classifier - Train accuracy: 1.0

Bagged classifier - Test accuracy: 0.9552238805970149



Q8)

Pruned Tree

- Advantages: Highly interpretable, that is, able to read which feature splits lead to each prediction. Faster to train and predict.
- Disadvantages: A single tree can still be somewhat unstable (small changes in training data can lead to a different tree). Slightly lower training accuracy due to the regularisation imposed by pruning.

Bagged Classifier

- Advantages: Reduces variance by averaging over many bootstrapped trees. Generally more robust and achieves better (or equal) test accuracy than a single tree. The OOB score provides a convenient internal estimate of generalisation error.
- Disadvantages: Much less interpretable than decision trees. Slower to train and higher memory usage due to storing multiple trees.

4.1.2 Exercise 9 (CORE)

For the range of `n_estimators` provided below: - Fit the bagged trees and compute the training accuracy, test accuracy, and out-of-bag scores. - Plot all three metrics as a function of the number of estimators. - Comment on the results and suggest a suitable number of estimators for this data.

```
[22]: n_est = np.array([1,5,10,20,50],dtype=int)

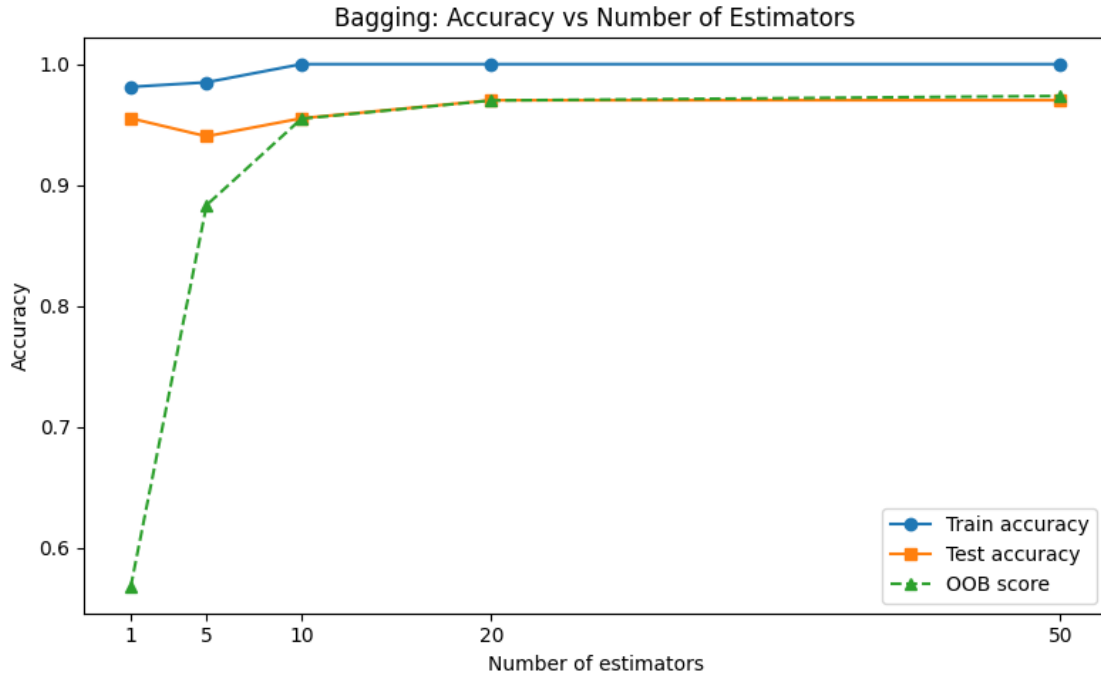
import warnings
warnings.filterwarnings('ignore')

train_acc, test_acc, oob_scores = [], [], []

for n in n_est:
    bag = BaggingClassifier(n_estimators=n, oob_score=True, random_state=42)
    bag.fit(X_train, y_train)
    train_acc.append(bag.score(X_train, y_train))
    test_acc.append(bag.score(X_test, y_test))
    oob_scores.append(bag.oob_score_)

# Plot
plt.figure(figsize=(8, 5))
plt.plot(n_est, train_acc, marker='o', label='Train accuracy')
plt.plot(n_est, test_acc, marker='s', label='Test accuracy')
plt.plot(n_est, oob_scores, marker='^', linestyle='--', label='OOB score')
plt.xlabel('Number of estimators')
plt.ylabel('Accuracy')
plt.title('Bagging: Accuracy vs Number of Estimators')
plt.legend()
plt.xticks(n_est)
plt.tight_layout()
plt.show()

results_df = pd.DataFrame({'n_estimators': n_est, 'train': train_acc,
                           'test': test_acc, 'OOB': oob_scores})
print(results_df.to_string(index=False))
```



n_estimators	train	test	OOB
1	0.981203	0.955224	0.567669
5	0.984962	0.940299	0.883459
10	1.000000	0.955224	0.954887
20	1.000000	0.970149	0.969925
50	1.000000	0.970149	0.973684

Q9)

- Training accuracy is consistently high across all ensemble sizes, since individual unpruned trees fit the training data closely.
- Both test accuracy and OOB score improve as the number of estimators increases, and largely plateau by around `n_estimators = 10`.
- Beyond 10 trees, the gains in accuracy are marginal, while computational cost continues to grow linearly.
- The OOB score tracks the test accuracy closely, confirming it is a reliable internal estimator of generalisation performance without using the held-out test set.
- A suitable choice for this dataset is around 10 estimators, which provides a good balance between accuracy and efficiency.

4.1.3 Bagging: low diversity in estimators

Bagging alone may not produce very diverse trees, and thus may result in only modest improvements in accuracy. In particular, if there is a strong set of predictors in the data, then the bagged trees will look quite similar to each other and predictions will be highly correlated. Averaging highly correlated quantities does not lead to a large reduction in variance and provides on very small

improvements in predictions.

Run the following code below, the print out the feature and threshold used for the first split for each of the trees in our ensemble. Notice how most of trees use the same first splitting rule.

```
[23]: # print out the first split for all trees in the bagged model
      for i in range(10):
          print('Estimator: ', i+1,
                ', First split feature: ', bc.estimators_[i].tree_.feature[0],
                ', First split threshold: ', bc.estimators_[i].tree_.threshold[0])
```

```
Estimator:  1 , First split feature:  2 , First split threshold:  207.0
Estimator:  2 , First split feature:  1 , First split threshold:
16.449999809265137
Estimator:  3 , First split feature:  2 , First split threshold:  207.5
Estimator:  4 , First split feature:  1 , First split threshold:
16.449999809265137
Estimator:  5 , First split feature:  1 , First split threshold:  16.5
Estimator:  6 , First split feature:  2 , First split threshold:  207.5
Estimator:  7 , First split feature:  2 , First split threshold:  207.5
Estimator:  8 , First split feature:  2 , First split threshold:  207.5
Estimator:  9 , First split feature:  0 , First split threshold:
42.30000114440918
Estimator: 10 , First split feature:  2 , First split threshold:  207.0
```

4.2 Random Forests

Random forests is one of the most widely used machine learning algorithms. A random forest is an ensemble that fits a number of decision tree classifiers on various random subsets of the data (Random Patches) and uses averaging to improve the predictive accuracy and control overfitting. Random forests are essentially bagged tree classifiers, but improve the diversity and decorrelate the trees compared with bagging by using a random sample of features each time a split in a tree is considered. The random forest algorithm can therefore be summarized as:

- Draw a random bootstrap sample of the training data.
- Grow a decision tree from the bootstrap data. At each node:
 - Randomly select a subset of features without replacement (defaults to the square root of the total number of features).
 - Split the node using the feature and threshold that provides the best split according to the objective function.
- Repeat the steps above for the desired number of bootstrapped datasets (estimators in the ensemble).
- Aggregate the predictions (`sklearn's RandomForestClassifier` aggregates the prediction probabilities).

4.2.1 Parameters

Important parameters include:

- **max_features**: specifies that number of random features to consider at each split. If set to the total number of features, then it is simply bagging.
- **n_estimators**: number of trees in the ensemble; larger is generally better as averaging more trees will yield a more robust ensemble.
- **max_samples**: sample size of the bootstrap dataset (useful when the training set is large to reduce memory and computational cost).

By default, large and complex trees are grown for each bootstrapped data. While these trees tend to have high variance and overfit to the training data, aggregation reduces variance and improves predictive accuracy. If desired, the complexity of the trees can also be limited by setting the parameters of the decision trees (e.g. **max_depth**, **max_leaf_nodes**); again this may be useful to reduce memory and computational cost.

4.2.2 Exercise 10 (CORE)

- Fit a random forest classifier that includes 10 trees in the ensemble.
- Print the splitting rule (feature and threshold) for each estimator. How does this compare to the bagged model?

```
[24]: from sklearn.ensemble import RandomForestClassifier

rf_10 = RandomForestClassifier(n_estimators=10, random_state=42)
rf_10.fit(X_train, y_train)

feature_names = list(X_train.columns)

print("Random Forest - first split for each of the 10 trees:")
for i, est in enumerate(rf_10.estimators_):
    feat_idx = est.tree_.feature[0]
    threshold = est.tree_.threshold[0]
    feat_name = feature_names[feat_idx] if feat_idx >= 0 else "leaf"
    print(f" Estimator {i+1:2d}: feature = {feat_name:20s} threshold = ␣
↪{threshold:.3f}")

print(f"\nRF (10 trees) - Train: {rf_10.score(X_train, y_train):.4f} | Test:␣
↪{rf_10.score(X_test, y_test):.4f}")
```

Random Forest - first split for each of the 10 trees:

Estimator 1:	feature = bill_depth_mm	threshold = 16.450
Estimator 2:	feature = flipper_length_mm	threshold = 202.500
Estimator 3:	feature = flipper_length_mm	threshold = 207.500
Estimator 4:	feature = bill_depth_mm	threshold = 16.450
Estimator 5:	feature = bill_depth_mm	threshold = 16.500
Estimator 6:	feature = flipper_length_mm	threshold = 207.500
Estimator 7:	feature = bill_length_mm	threshold = 42.350
Estimator 8:	feature = bill_depth_mm	threshold = 16.450
Estimator 9:	feature = bill_length_mm	threshold = 42.300
Estimator 10:	feature = bill_depth_mm	threshold = 16.450

RF (10 trees) - Train: 1.0000 | Test: 0.9701

Q10)

Bagged model: nearly all 10 trees used the same feature (`flipper_length_mm`) and a very similar threshold for the first split, reflecting the strong dominance of this feature. The bootstrapped datasets all contain it and it is always selected as the best split.

Random forest: The first split is more diverse across the 10 trees, as only a random subset of features is considered at each split (default: $\sqrt{D} = \sqrt{4} = 2$ features). This decorrelation of the trees is the key advantage of random forests over plain bagging: the trees are more diverse, so averaging them provides a larger reduction in variance and typically better generalisation.

4.2.3 Exercise 11 (CORE)

- Fit a random forest to your data, using the OOB error to tune the number of features considered at each split, for `max_features` in [2,3,4]. What is the optimal number of features?
- Visualize the confusion matrix on the train and test data.

```
[25]: # Note: let rf be the name your random forest object with the optimal
      ↪ max_features or change the name in exercise 12

max_features_range = [2, 3, 4]
oob_results = {}

for mf in max_features_range:
    rf_tmp = RandomForestClassifier(n_estimators=100, max_features=mf,
                                   oob_score=True, random_state=42)
    rf_tmp.fit(X_train, y_train)
    oob_results[mf] = rf_tmp.oob_score_
    print(f"max_features={mf}: OOB score = {rf_tmp.oob_score_:.4f}")

best_mf = max(oob_results, key=oob_results.get)
print(f"\nOptimal max_features: {best_mf} (OOB = {oob_results[best_mf]:.4f})")

# Fit final RF with the best max_features
rf = RandomForestClassifier(n_estimators=100, max_features=best_mf,
                           oob_score=True, random_state=42)
rf.fit(X_train, y_train)

print(f"Train accuracy: {rf.score(X_train, y_train):.4f}")
print(f"Test accuracy: {rf.score(X_test, y_test):.4f}")

# Confusion matrices
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
ConfusionMatrixDisplay.from_estimator(rf, X_train, y_train,
                                       ax=axes[0], colorbar=False)
axes[0].set_title(f"Random Forest (max_features={best_mf}) - Train")
```

```
ConfusionMatrixDisplay.from_estimator(rf, X_test, y_test,
                                     ax=axes[1], colorbar=False)
axes[1].set_title(f"Random Forest (max_features={best_mf}) - Test")
plt.tight_layout()
plt.show()
```

max_features=2: OOB score = 0.9850

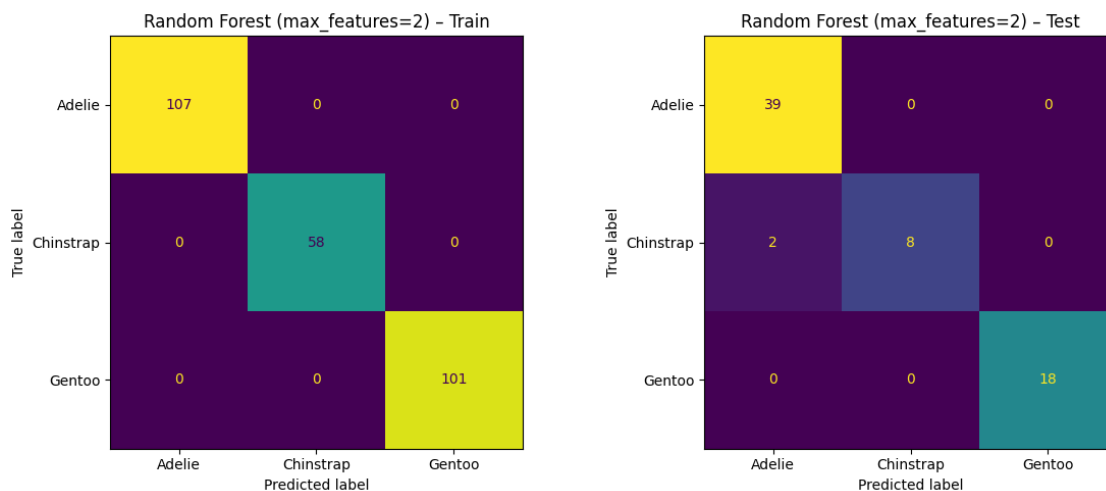
max_features=3: OOB score = 0.9774

max_features=4: OOB score = 0.9774

Optimal max_features: 2 (OOB = 0.9850)

Train accuracy: 1.0000

Test accuracy: 0.9701



4.2.4 Feature Importance

The fitted `RandomForestClassifier` also has an attribute `feature_importances_`, that reflects the relative rank (i.e. depth) and predictive power of a feature, averaged across all trees in the ensemble. For further details, see <https://scikit-learn.org/stable/modules/ensemble.html#feature-importance-evaluation>, and for an example, see https://scikit-learn.org/stable/auto_examples/ensemble/plot_forest_importances.html.

Let's start by visualizing the feature importances for our pruned tree in exercise 7 (note: replace `clf` with the name of your tree from exercise 7, if not already called `clf`). Notice how this reflects the depth of the feature and impurity decrease.

```
[26]: # NOTE: change clf to name of your pruned tree in exercise 7

# Extract feature names
```

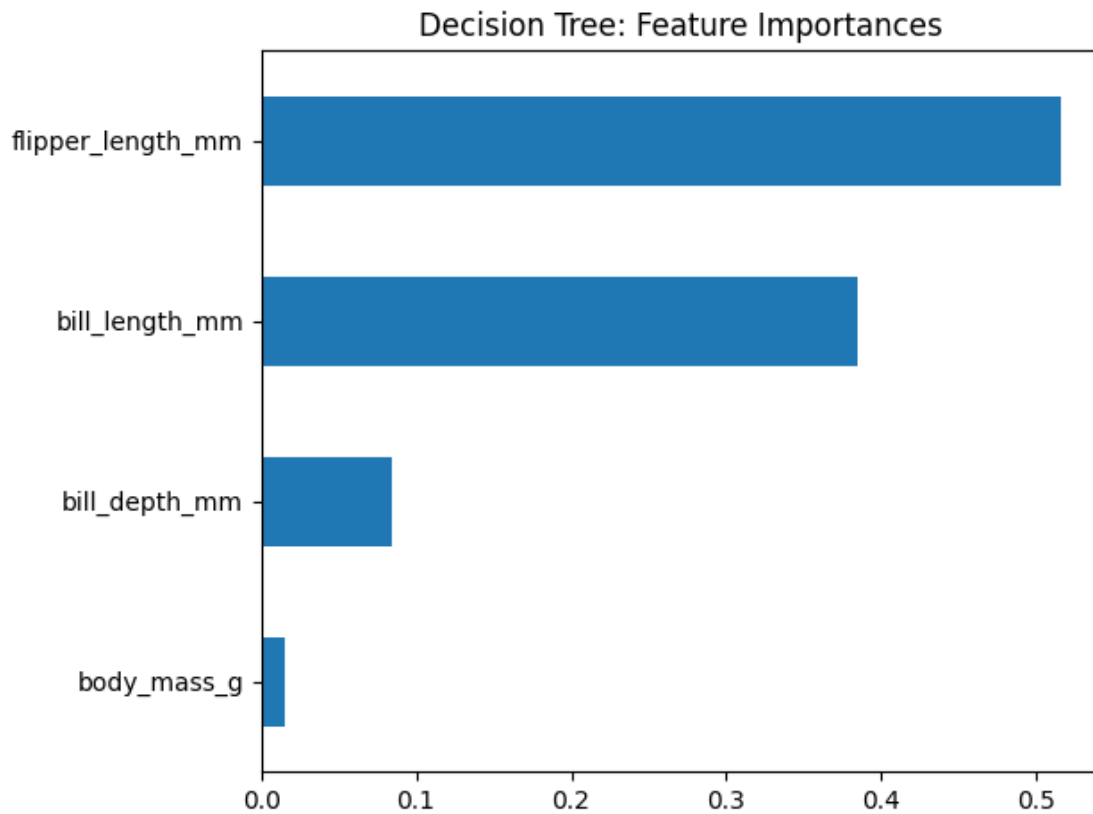
```

feature_names = clf.feature_names_in_

# Create a pandas series with the feature importances
importances = pd.Series(
    clf.feature_importances_, index=feature_names
).sort_values(ascending=True)

# Plot the feature importances
ax = importances.plot.barh()
ax.set_title("Decision Tree: Feature Importances")
ax.figure.tight_layout()

```



4.2.5 Exercise 12 (CORE)

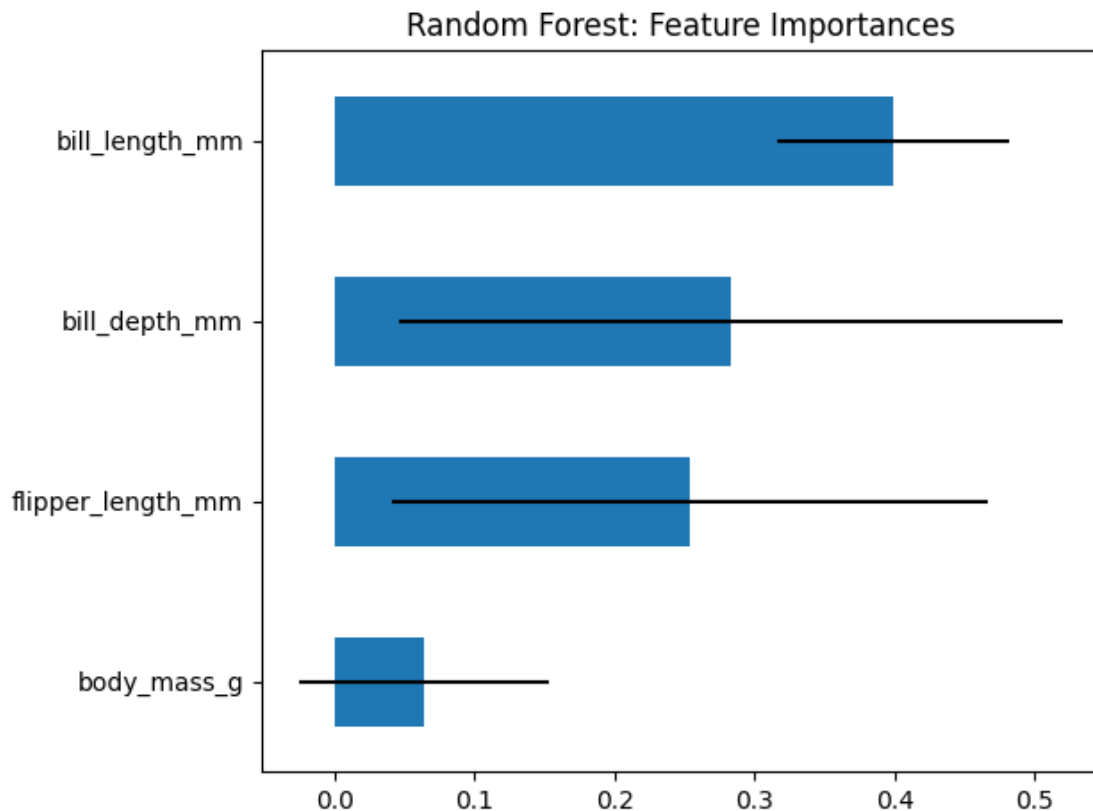
- Run the following code to plot the feature importance (averaged across all trees along with the standard deviation).
- Visualize the correlation matrix of the features.
- Has the ranking of the features changed comparing the pruned tree to the random forest? Does the correlation matrix provide any insight on why?

```
[27]: # Note: change rf to the name your random forest object in exercise 12

# Compute the standard deviation
std = np.std([tree.feature_importances_ for tree in rf.estimators_], axis=0)

# Create a pandas series with the feature importances
importances_rf = pd.Series(
    rf.feature_importances_, index=feature_names
).sort_values(ascending=True)

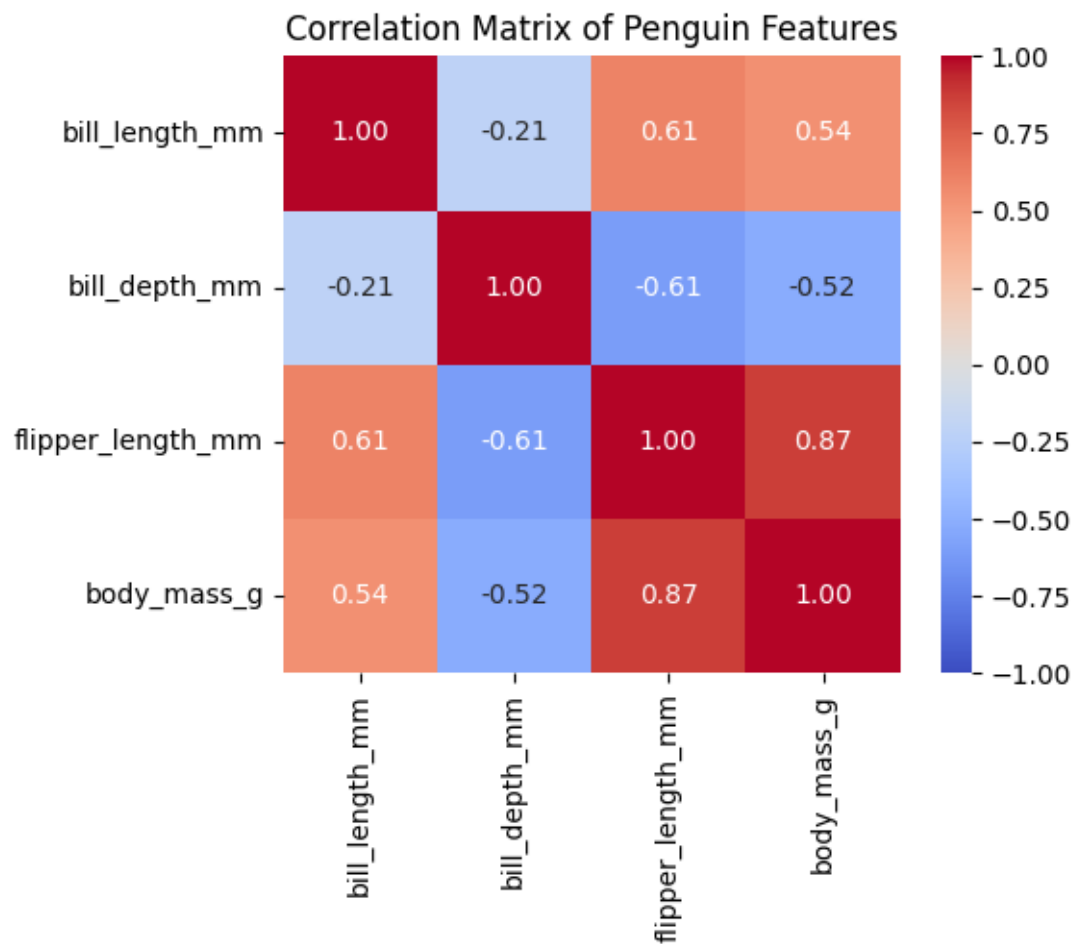
# Plot the feature importances
ax = importances_rf.plot.barh(xerr=std)
ax.set_title("Random Forest: Feature Importances")
ax.figure.tight_layout()
```



```
[28]: corr = X_train.corr()

fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
            vmin=-1, vmax=1, ax=ax, square=True)
```

```
ax.set_title("Correlation Matrix of Penguin Features")
plt.tight_layout()
plt.show()
```



Q12)

Feature importance ranking

- Pruned single tree: `flipper_length_mm` dominates the importance ranking, because it is used at the very first (root) split and alone accounts for most of the impurity reduction. The other features appear much less important by comparison.
- Random forest: Importance is more evenly distributed across features, particularly `bill_length_mm`, `bill_depth_mm`, and `flipper_length_mm`.

Correlation matrix

- `flipper_length_mm` and `body_mass_g` are strongly positively correlated (~ 0.87). In a single tree, once `flipper_length_mm` is used at the root, `body_mass_g` carries little additional discriminating power and is rarely selected.

- In the random forest, `flipper_length_mm` is sometimes excluded from the candidate feature set at a given node (only $\sqrt{D}$ features are considered). In those cases, `body_mass_g` or other correlated features are used instead. Over many trees, this means correlated features share the importance more equally, giving a more balanced and reliable picture of feature relevance.

4.3 Voting Classifier

While bagging and random forests focused on creating an ensemble of the same estimator (e.g. trees) fit on different random subsets of the data, the idea behind **voting** is to combine different estimators. The predictions then can be aggregated either by using a majority vote or by averaging predicted probabilities (soft vote). This can be useful for a set of equally well performing models in order to balance out their individual weaknesses.

In the `VotingClassifier`, the default is to aggregate predictions with **hard voting** (i.e. `voting='hard'` by default), which simply looks at the tally of votes received by each class across the estimators in the ensemble. Instead, **soft voting** (i.e. `voting='soft'`) averages the predicted probabilities across the estimators in the ensemble.

For further details, please see the documentation from here: <https://scikit-learn.org/stable/modules/ensemble.html#voting-classifier>

4.3.1 Exercise 13 (EXTRA)

- Create a voting classifier that combines your pruned tree, random forests, logistic regression, and support vector classifier.
- How does the accuracy on the test data compare across each model individually and the voting classifier?

```
[29]: from sklearn.ensemble import VotingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

# Note: LogisticRegression and SVC benefit from scaled features
lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000,
↳ random_state=42))
svc = make_pipeline(StandardScaler(), SVC(probability=True, random_state=42))
↳ # probability=True for soft voting

estimators = [
    ('pruned_tree', clf),
    ('random_forest', rf),
    ('logistic_regression', lr),
    ('svc', svc),
]

# Hard voting
vc_hard = VotingClassifier(estimators=estimators, voting='hard')
```

```

vc_hard.fit(X_train, y_train)

# Soft voting (averages predicted probabilities)
vc_soft = VotingClassifier(estimators=estimators, voting='soft')
vc_soft.fit(X_train, y_train)

# Compare individual and ensemble test accuracies
print("Test Accuracy Comparison")
for name, est in estimators:
    est.fit(X_train, y_train) # ensure fitted
    acc = est.score(X_test, y_test)
    print(f" {name:25s}: {acc:.4f}")

print(f" {'Voting (hard)':25s}: {vc_hard.score(X_test, y_test):.4f}")
print(f" {'Voting (soft)':25s}: {vc_soft.score(X_test, y_test):.4f}")

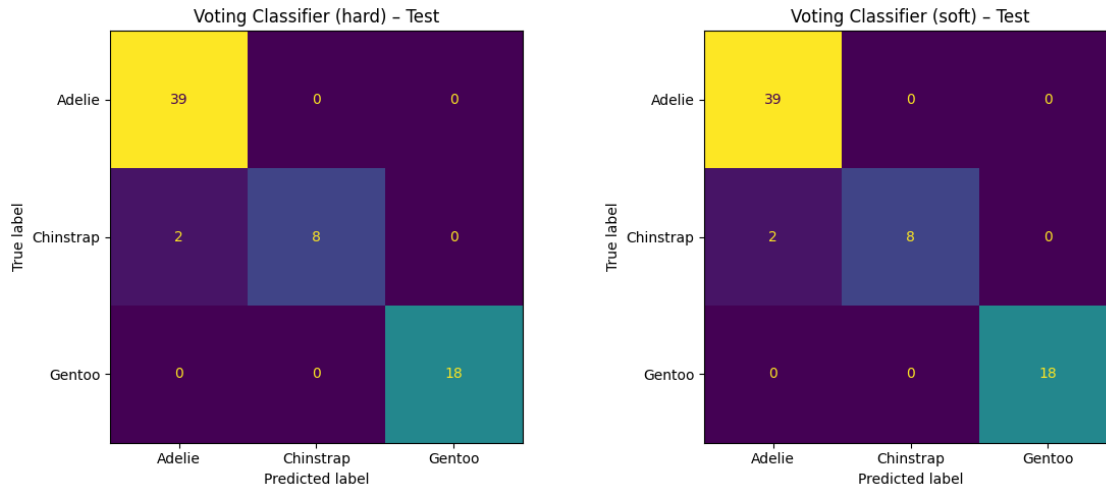
# Confusion matrices for the two voting classifiers
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
ConfusionMatrixDisplay.from_estimator(vc_hard, X_test, y_test,
                                     ax=axes[0], colorbar=False)
axes[0].set_title("Voting Classifier (hard) - Test")

ConfusionMatrixDisplay.from_estimator(vc_soft, X_test, y_test,
                                     ax=axes[1], colorbar=False)
axes[1].set_title("Voting Classifier (soft) - Test")
plt.tight_layout()
plt.show()

```

Test Accuracy Comparison

pruned_tree	: 0.9403
random_forest	: 0.9701
logistic_regression	: 0.9701
svc	: 0.9701
Voting (hard)	: 0.9701
Voting (soft)	: 0.9701



Q13)

- All four individual classifiers achieve high test accuracy on this well-separated dataset.
- Both the hard-voting and soft-voting ensembles achieve accuracy at least as high as the best individual model.
- Soft voting tends to outperform hard voting when the component classifiers provide well-calibrated probability estimates, since it uses more information than a simple majority tally.
- The voting classifier is most beneficial when the individual classifiers make different kinds of errors, causing each model's weaknesses to be compensated by the others. On a clean, low-dimensional dataset like this one, the gains are modest, but on more complex problems, the ensemble can provide a meaningful improvement.
- The main downside of the voting classifier is the added complexity: four models must be trained, tuned, and maintained, making the pipeline less interpretable than a single decision tree.

5 Competing the Worksheet

At this point you have hopefully been able to complete all the CORE exercises and attempted the EXTRA ones. Now is a good time to check the reproducibility of this document by restarting the notebook's kernel and rerunning all cells in order.

Before generating the PDF, please **change 'Student 1' and 'Student 2' at the top of the notebook to include your name(s)**.

Once that is done and you are happy with everything, you can then run the following cell to generate your PDF. Once generated, please submit this PDF on Learn page by 16:00 PM on the Friday of the week the workshop was given.

```
[30]: # !jupyter nbconvert --to pdf mlp_week08.ipynb
```